/gcf

Toggle GCF encoding of tool results: /gcf on|off|show|help

Overview

`/gcf` turns the GCF (Graph Compact Format) tool-result encoding feature on or off. When enabled, JSON-dense tool results are re-encoded in the LLM view as a lossless `[BEGIN GCF generic]` … `[END GCF]` block, saving tokens without dropping any values.

The encoding applies only to the model-facing copy of a tool result. TUI rendering, the activity log, memory extraction, PostToolUse hooks, compaction serialisers, and the persisted session history all keep seeing the raw JSON; only the observation handed to the LLM is re-encoded (the FR-005 routing map in `crates/ragent-agent/src/session/history.rs` `tool_result_content_for_llm`).

While GCF is on, the agent system prompt also gains a short `## GCF Encoding Primer` section teaching the model how to read the blocks.

The encoding itself is provided by the published `gcf` crate (v3.0.1, generic profile: `encode_generic` / `decode_generic`). The block wrapper and eligibility rules live in the `tool_result_content_for_llm` hook in `crates/ragent-agent/src/session/history.rs`.

Eligibility rules

A tool result is encoded only when all of the following hold:

- the feature is enabled (`gcf.enabled` or `/gcf on`);
- the observation is at least 200 characters (`GCF_MIN_JSON_CHARS`);
- it parses as a JSON object or array (plain prose and error text never encode);
- the `gcf` generic profile encodes it to a block at least 10% smaller than the raw JSON including the `[BEGIN...]`/`[END...]` markers (`GCF_MIN_SAVINGS_PERCENT`);
- the resulting block fits the 12000-character LLM tool-result budget, so it is never truncated mid-payload (the lossless guarantee, FR-007).

Any other case falls back to the raw observation unchanged; encode failures never break a tool result. The `wait_agents` and `list_agents` tools are exempt from GCF entirely (their results take the sub-agent report path).

Syntax

```
/gcf              Print the help table (same as /gcf help)
/gcf on           Enable GCF encoding and persist it
/gcf off          Disable GCF encoding and persist it
/gcf show         Show the effective state and its source
/gcf help         Print the help table
```

`--help` and `-h` are accepted as aliases of `help`. Unknown subcommands print the usage text.

Options / Subcommands

Form	Description
`on`	Enable GCF encoding via `ragent_config::gcf::persist_gcf(true)`, then invalidate the session processor's config cache so the next turn picks up the saved file (FR-002).
`off`	Disable GCF encoding the same way.
`show`	Report the effective runtime flag and where the state came from (see Output).
`help`	Print the help table (bare `/gcf` does the same).

If persisting fails (config file not writable), the in-memory flag still applies for the live session and the status line shows `gcf: on (unsaved)`. The state persists across restarts via the `gcf.enabled` key in `ragent.json`; the key is omitted from the config file while the feature is at its default-off state.

Examples

`/gcf on`

Enables encoding and writes `gcf.enabled: true` to the active config source.

`/gcf off`

Disables encoding and removes the `gcf` section from the config file.

`/gcf show`

Prints the current state plus its source, for example `Source: persisted config (gcf.enabled = true)` or `Source: default-off (nogcfsection in the config file)`.

`/gcf help`

Prints the help table.

Output

`on`/`off` confirm the change:

```
GCF encoding of tool results enabled (persisted to the config file).
```

A failed persist surfaces as a warning:

```
GCF encoding enabled for this session, but saving the config failed: <e> (unsaved).
```

`show` prints two lines: `GCF encoding of tool results: **on**` and a `Source:` line of one of `in-session change (unsaved)` / `persisted config (gcf.enabled = true|false)` / `default-off (nogcfsection in the config file)`.

With GCF on, an eligible tool result reaches the model as an illustrative block of this shape:

```
[BEGIN GCF generic]
GCF profile=generic
count=3
ok=true
tags[2]: a,b
```

```
[END GCF]
```

The payload is lossless: every value is recoverable, nothing is omitted or summarised.

Related

- `## GCF Encoding Primer` in the system prompt (added while GCF is on).
- `/tools` - controls which tool families are visible, unrelated to encoding.
- `/compact` - compaction serialisers read raw JSON, never GCF blocks.